TRUE 贞实科技

AI 赋能控制

指令
功能手册

V1.7.4

2026.04

文档版本

版本号	修订日期
1.3	2025 年 08 月 21 日
1.4	2025 年 08 月 30 日
1.5	2025 年 09 月 18 日
1.6	2025 年 10 月 14 日
1.7	2025 年 10 月 28 日
1.7.1	2025 年 12 月 15 日
1.7.2	2025 年 12 月 30 日
1.7.3	2026 年 01 月 16 日
1.7.4	2026 年 01 月 22 日
1.7.5	

目录

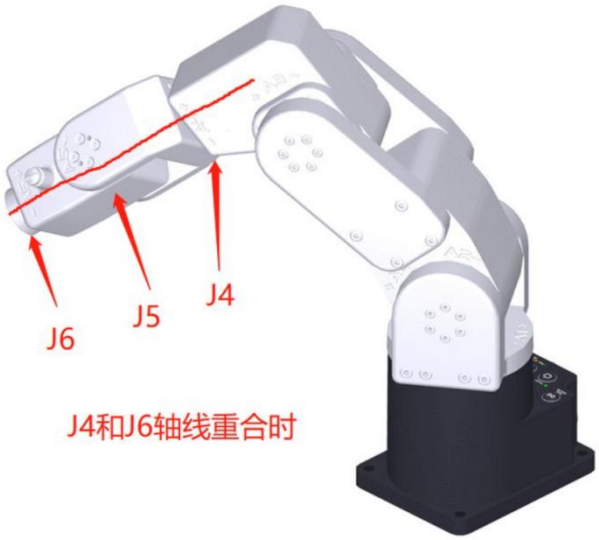
文档版本 II
常见奇异点说明 1
1.Wrist singularity（手腕奇异点） 1
2.Elbow singularity（肘关节奇异点） 1
3.Shoulder singularity（肩部奇异点） 2
一、常用变量类型 3
1. load 变量 3
2. jointtarget 关节空间参数 3
3. robottarget 笛卡尔空间参数 4
4. speed 速度参数 4
5. zone 转弯半径 5
6. tool 工具参数 6
7. wobj 基坐标系参数 7
二、常用变量指令 8
1. Var 定义变量 8
三、基础运动指令 9
1. Enable 上使能 9
2.Disable 下使能 10
3.Recover 状态同步 11
4.Start 开始 12
5.Stop 停止 12
6.Pause 暂停 13
7. JogJoint 关节点动 14
8.MoveAbsJ 关节运动 15
9. JogCartesian 笛卡尔点动（位置+姿态） 16
10. SpeedL 给定速度（在线规划） 17
四、在线规划指令 18
1. SpeedJ 关节给定速度（在线规划） 18
五、常用系统指令 19
1.Clear 错误清理 19
2.SetTool 设置工具 19

3.SetWobj 设置基坐标系 21
4.SetUsingSP 最优规划求解 22
六、高级规划指令 23
1.MoveBlend 笛卡尔空间下轨迹融合 23
2.MoveBlendScurve 笛卡尔空间下轨迹融合（匀速） 25
3.MoveBlendOptimalpos 笛卡尔空间下轨迹融合（最大速） 27
4.MoveS 笛卡尔空间下轨迹融合（全程） 28
5.ServoSeries 关节空间下轨迹融合指令 29
6. MoveSeriesToppJ2 关节空间下轨迹融合（最大速） 30
七、力控指令 31
1. DragFtInCartesian 笛卡尔空间下力-位混合控制 31
八、IO 模块指令 32
1.GetDI 读取输入电平状态 32
2.SetDO 设置输出电平状态 33
3.PulseDO 输出数字脉冲信号 34
4.AddIO 注册新的 I/O 信号输出口 35

常见奇异点说明

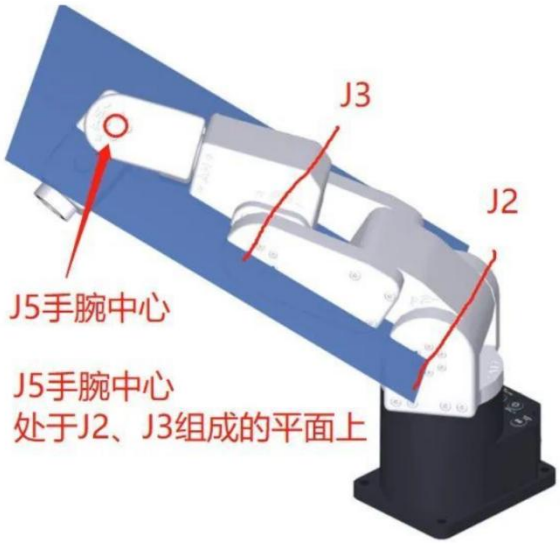
1.Wrist singularity（手腕奇异点）

手腕奇异点发生在 J4 轴和 J6 轴重合时，也就是 J5 轴为 0°时，手腕奇异点是关节机器人最容易遇到到奇异点。



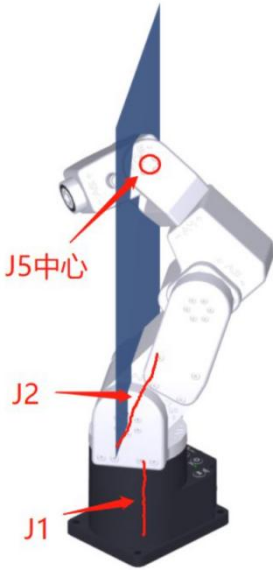
2.Elbow singularity（肘关节奇异点）

肘关节奇异点发生在 J5 中心点处于 J1、J2 组成的平面上。
多数机器人的大臂不会完全伸直，因此一般不会碰到肘关节奇异点。



3.Shoulder singularity（肩部奇异点）

肩部奇异点发生在 J5 中心点处于通过 J1、平行 J2 轴的平上时。
肩部奇异点非常复杂，机器人逆运算时存在无数解。



一、常用变量类型

1. load 变量

➢ 变量说明

该结构体包含一个刚体在某一个坐标系下的十个描述质量分布特性的 double 型参数：mass 质量，cogx 质心在 x 方向的偏移量，cogy 质心在 y 方向的偏移量，cogz 质心在 z 方向的偏移量，i 表示惯性张量矩阵参数(ixx,iyy,izz,ixy,ixz,iyz)。（质量单位：千克 kg；偏移量单位：米 m）

对于机器人来说，该参数可用于描述机器人末端负载参数特性，坐标系一般默认为末端法兰盘坐标系

➢ 使用示例

Var --name=piece --type=load --value={mass, mass*cogx, mass*cogy,mass*cogz,0,0,0,0,0,0}

2. jointtarget 关节空间参数

➢ 变量说明

该结构体共有十个 double 型参数（J0~J9），存储机器人目标关节角度（单位是弧度）和外部轴位置（单位是米或弧度）。会从该十个数依次匹配目标关节角度与外部轴位置。目标关节角度与外部轴总数不得超过十个。

➢ 使用示例

Var --name=home --type=jointtarget --value={J0,J1,J2,J3,J4,J5,J6,J7,J8,J9}

3. robottarget 笛卡尔空间参数

➢ 变量说明

该结构体储笛卡尔空间的目标位置及姿态（double），包含 x, y, z, q1, q2, q3, q4，适用于 MoveJ, MoveL，MoveC 指令的，位置单位为米。（q1,q2,q3,q4 分别对应 x,y,z,w）

➢ 使用示例

Var --name=p1 --type=robottarget --value={x,y,z,q1,q2,q3,q4}

4. speed 速度参数

➢ 变量说明

该结构体包含 5 个 double 型成员变量，per 关节速度百分比，适用于 MoveAbsJ 和 MoveJ。tcp 线速度，eri 空间旋转速度，exj_l 外部轴线速度，exj_r 外部轴角速度。示例：speed v1={40 300 100 200 1000}

➢ 使用示例

Var --name=v1 --type=speed --value={per,tcp,ori,exj_r,exj_l}

在 xml 文件的 model-Variable 节点下，有一些已定义好的 speed 结构体，例如 v100={0.1,0.1,3.4906585,0,0}，其中第一个 0.1 表示关节 max_vel 的 10%。

5. zone 转弯半径

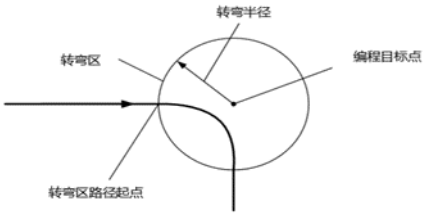
➢ 变量说明

该结构体变量用于定义某一个运动如何结束或者说定义两条运动轨迹之间转弯区的大小。该结构体包含两个 double 型变量，distance 为笛卡尔空间的转弯区大小，适用于 MoveLL 指令，单位是毫米；percent 为转弯区所占运动时间的百分比，适用于 MoveJ、MoveL、MoveC 等运动指令直接的融合

转弯区的距离大小不能超过前后两条待融合路径长度的一半，如果超过，系统会自动将转弯区长度缩小到前后两条路径中长度较小的一半；转弯区的时间不能超过前后两条待融合运动时间的一半，如果超过，系统会自动将转弯区时间缩小到前后两条路径中时间较小的一半。使用转弯区可以避免机器人频繁启停，显著减少节拍时间。

➢ 使用示例

Var --name=z100 --type=zone --value={dis,per}



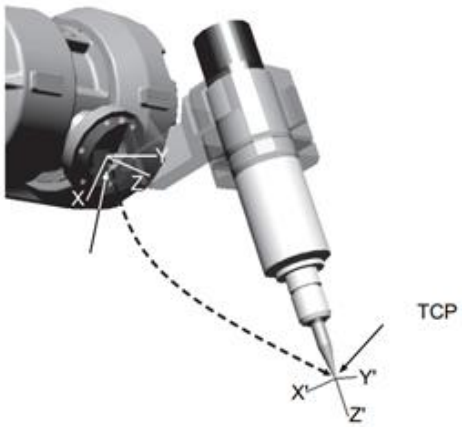
6. tool 工具参数

➢ 变量说明

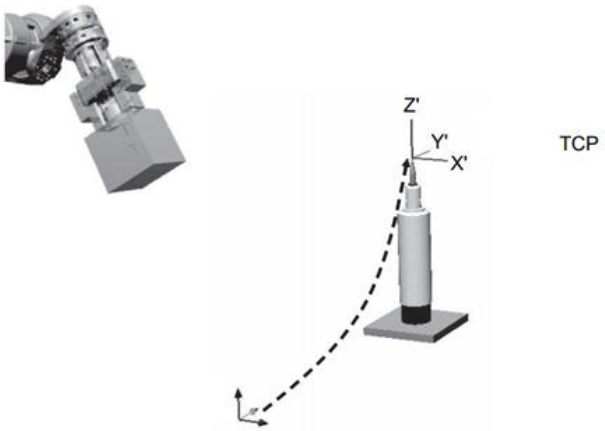
该结构体记录工具参数，包含机器人所用工具的 TCP、姿态，具体包含以下三个子结构体： pose 类型，包含 TCP 相对于机器人末端法兰盘坐标系的 x,y,z 三个方向的偏移量，单位为毫米，以及工具坐标系相对于法兰坐标系的姿态偏移量，用四元数表示。pos 结构体如下：pose pose1={x,y,z,q1, q2, q3, q4}。

➢ 使用示例

Var --name=tool --type=tool --value={true,x,y,z,q1,q2,q3,q4}



如果使用固定工具，则相对于世界坐标系来定义工具坐标系。



xx1100000518

Figure 3.4: 固定工具

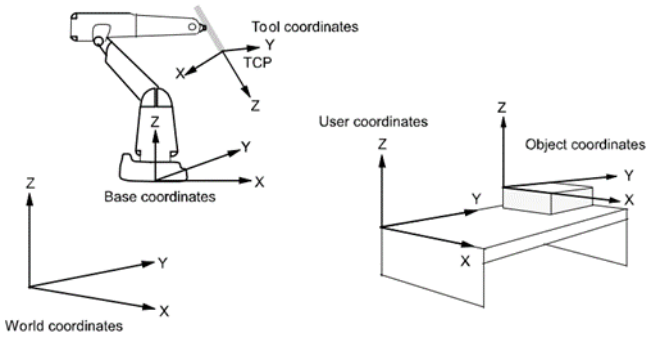
7. wobj 基坐标系参数

➢ 变量说明

该结构体所有运动指令中使用的位置都是在工件坐标系下定义的（如果没有指定工件坐标系，则默认 在世界坐标系下定义，世界坐标系可以被看做是 wobj0）。

➢ 使用示例

Var --name=wobj2 --type=wobj --value ={x, y, z, q1, q2, q3, q4}



二、常用变量指令

1. Var 定义变量

➢ 指令用法	定义变量

➢ 指令参数	数据类型	含义
--name	String	变量名
--type	String	变量类型
--value	Number/Matrix	变量值
--degree_unit	Number	输入类型
--clear	None	清除所有已定义变量

➢ **指令说明**

1）当其他指令参数需要调用这些变量时，可通过 --type_var 的方式调用；

2）--type 支持的变量类型有 Number,Matrix,String,pos,orient,pose,speed,robottarget,zone,jointtarget,tool,wobj;

3）--value 可以是单个值，也可以是用大括号包含的矩阵;

4）var 为临时变量，重启后不保存，使用方式为 _var ,如： --jointtarget_var 、--robottarget_var 、--speed_var 等等（见运动指令中的使用示例）。

5）在定义关节变量（jointtarget）时，可以通过 degree_unit=1 来选择 value 单位为角度，degree_unit=0 则 value 单位为弧度。

6）变量名称不支持纯数字，需要以英文字母开头

➢ **指令示例**

```
Var --name=pi --type=Number --value=3.14
Var --name=pe --type=Matrix --value={0.3,0.2,0.1,0,0,0}
Var --name=z2 --type=zone --value={0.1,0.1}
Var --type=Matrix --name=m0 --value={1,1,1,0.1,0.1,0.1}
Var --type=robottarget --name=pq --value={0,0,0,0,0,0,0}
Var --type=jointtarget --name=j0 --value={0.1,-0.1,-0.1,-0.1,0.1,0.4,0,0,0,0} --degree_unit=1
Var --type=tool --name=toolPP --value={0.1,0,0,0,0,0,1}
Var --type=load --name=load1 --value={0,0,0,0,0,0,0,0,0,0}
Var --clear    //清除所有 var 变量
```

三、基础运动指令

1. Enable 上使能

➢ **指令用法**

使能所有的伺服电机设备或者某一个伺服电机设备

➢ **指令参数**

	数据类型	含义
-- limit_time	num	容许的最大等待时间,单位为毫秒。当在该时间内未能使能成功，则系统会抛出异常，错误等级为 ERROR。
--load/--load_var	load	指定机器人末端负载。
--all/--motion_id	num	--all 表示对所有电机使能；--motion_id=i 时表示对第 i+1 个电机使能。

➢ **指令说明**

➢ **指令示例**

例 1

Enable：相当于 Enable --all --limit_time=5000 --load=load0，或 Enable -a --limit_time --load=load0。功能为使能全部电机，使能超时时间为 5000ms，末端负载编号为 xml 中保存的参数 load0。

例 2

Var --type=load --name=load1 --value={0,0,0,0,0,0,0,0,0,0}

Enable --motion_id=0 --limit_time=4000 --load_var=load1。功能为使能第一个电机，使能超时时间为 4000ms，负载为定义的变量 load1。

2.Disable 下使能

➢ **指令用法**

下使能所有的伺服电机设备或者某一个伺服电机设备

➢ **指令参数**

	数据类型	含义
-- limit_time	num	容许的最大等待时间,单位为毫秒。当在该时间内未能使能成功，则系统会抛出异常，错误等级为 ERROR。
--load/--load_var	load	指定机器人末端负载。
--all/--motion_id	num	--all 表示对所有电机使能；--motion_id=i 时表示对第 i+1 个电机使能。

➢ **指令说明**

➢ **指令示例**

例 1

Disable：相当于 Disable --all --limit_time=5000 --load=load0，或 Disable -a --limit_time --load=load0。功能为使能全部电机，使能超时时间为 5000ms，末端负载编号为 xml 中保存的参数 load0。

例 2

Var --type=load --name=load1 --value={0,0,0,0,0,0,0,0,0,0}

Disable --motion_id=0 --limit_time=4000 --load_var=load1。功能为使能第一个电机，使能超时时间为 4000ms，负载为定义的变量 load1。

3.Recover 状态同步

➢ 指令用法	同步模型状态与实际被控对象的状态
➢ 指令参数	无
➢ 指令说明	从 CST 模式切换到 CSP 模式前，要使用该指令，用于同步实际模型状态，不然无法通过 Show 指令获取机器人当前实际状态。

➢ 指令示例

无

4.Start 开始

➢ 指令用法	将被暂停或停止的运动重新启动

➢ 指令参数	数据类型	含义
--last_count	num	运动重启到位所需的控制周期数目

➢ 指令说明	无

➢ 指令示例

例 1

Start：相当于 Start --last_count=10。花费 10 个控制周期完成运动重启。

Start --last_count=5：花费 5 个控制周期完成运动重启。

5.Stop 停止

➢ 指令用法	停止运动，并跳出该指令。该指令后面的缓冲指令均不执行

➢ 指令参数	数据类型	含义
--last_count	num	完全停止运动所需的控制周期数目

➢ 指令说明

1-可用于紧急停止机器人当前运动，可使用默认参数；
2-Stop 与 Start 一一对应；
3-多条指令阻塞调用时，一个 Stop 只能终止一条指令。

➢ 指令示例

例 1

Stop：相当于 Stop --last_count=10。花费 10 个控制周期完全停止运动。

Stop --last_count=5：花费 5 个控制周期完全停止运动。

6.Pause 暂停

➢ **指令用法**

暂停运动，不跳出该指令。Start 后该指令可继续执行。该指令后面的缓冲指令会继续执行

➢ **指令参数**

	数据类型	含义
--last_count	num	完全停止运动所需的控制周期数目

➢ **指令说明**

➢ **指令示例**

例 1

Pause：相当于 Pause --last_count=10。花费 10 个控制周期完全暂停运动。

Pause --last_count=5。花费 5 个控制周期完全暂停运动。

7. JogJoint 关节点动

➢ 指令用法	关节空间指定轴 Jog 运动

➢ 指令参数	数据类型	含义
--increase_count	Number	速度变量
--speed	Number	关节速度，单位为 rad/s
--motion_id	Number	
--direction	Number	运动方向：1-正方向运动，-1-负方向运动

➢ 指令说明	默认参数：--stop

➢ 指令示例

例 1

```
JogJoint --speed=v100 --motion_id=0 --increase_count=50 --direction=1
JogJoint --speed=v100 --motion_id=1 --increase_count=50 --direction=-1
JogJoint --speed=v100 --motion_id=5 --increase_count=50 --direction=1
```

8.MoveAbsJ 关节运动

➢ 指令用法	关节空间运动

➢ 指令参数	数据类型	含义
--jointtarget/ --jointtarget_var/ --jointtarget_value/	jointtarget	目标关节角度变量
--speed/ --speed_var --speed_value	Speed	速度变量
--zone/--zone_var	zone	轨迹融合变量

➢ 指令说明	无

➢ 指令示例

例 1

Var --type=jointtarget --name=j0 --value={0.1,0.2,0.3,0.4,0.5,0,0,0,0,0}

MoveAbsJ --jointtarget_var=j0：其他均取默认值

例 2

MoveAbsJ --jointtarget_value={0.1,0.2,0.3,0.4,0.5,0,0,0,0,0} --speed_value=0.1

9. JogCartesian 笛卡尔点动（位置+姿态）

➢ 指令用法	笛卡尔空间连续运动

➢ 指令参数	数据类型	含义
--increase_count	Number	加速过程周期数
--moving_type	Number	笛卡尔运动方式 0~5（X、Y、Z 平动、旋转）
-- speed --speed_value/	Number	六维笛卡尔速度：x/y/z 方向平动，单位 m/s，rx/ry/rz 方向转动，单位 rad/s。
--direction	Number	运动方向：1-正方向运动，-1-负方向运动
--coordinate	Number	运动的参考坐标系： 0-绝对世界坐标系，1-工具坐标系

➢ 指令说明

➢ 指令示例

例 1

```
JogCartesian --moving_type=0 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
JogCartesian --moving_type=1 --direction=-1 --speed_value=0.1 --coordinate=0 --increase_count=50
JogCartesian --moving_type=2 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
```

例 2

```
JogCartesian --moving_type=3 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
JogCartesian --moving_type=4 --direction=1 –speed_value=0.1 --coordinate=0 --increase_count=50
JogCartesian --moving_type=5 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
```

10. SpeedL 给定速度（在线规划）

➢ 指令用法	笛卡尔空间接受期望速度（可打断--在线规划）

➢ 指令参数	数据类型	含义
--last_count	Number	指令保留时间
--vel	Number	旋量速度
--acc	Number	旋量加速度
--jerk	Number	旋量加加速度
--coordinate	Number	选择基准坐标系为世界坐标系或工具坐标系

➢ 指令说明

用户设置一组笛卡尔期望速度，最大加速度以及最大加加速度以及运动时间，指令进行规划连续轨迹到达期望速度。

➢ 指令示例

例 1 中，设置了机器人在基坐标系下，沿 x 轴的正负方向，保持 100count。

例 2 中，设置了机器人在末端工具坐标系下，沿 Y 轴的正负方向，保持 100count。

例 1

```
SpeedL --vel={0.1,0,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=100 --coordinate=0
SpeedL --vel={-0.1,0,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=100 --coordinate=0
```

例 2

```
SpeedL --vel={0,0.1,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=100 --coordinate=1
SpeedL --vel={0,-0.1,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=100 --coordinate=1
```

四、在线规划指令

1. SpeedJ 关节给定速度（在线规划）

➢ 指令用法	关节空间在线接受期望速度

➢ 指令参数	数据类型	含义
--last_count	Number	指令保留时间
--vel	Number	期望关节速度，单位为 rad/s
--acc	Number	关节最大加速度
--dec	Number	关节最大减速度
--jerk	Number	关节最大加加速度

➢ 指令说明	默认参数: --stop（指令无后缀名则默认--stop，指令将速度减为 0 退出） 用户设置一组关节期望速度，最大加速度以及最大加加速度，指令实时规划连续轨迹到达期望速度。用户可以再次用本指令发送新的期望运动来实时更改关节运动。
➢ 指令示例	例 1 中，设置了机器人所有轴的期望速度 0.1rad/s，保持 1000count，然后等待 950ms，发送新的期望速度-0.1rad/s。 注意，若该指令使用同步的方式调用，则指令会运行完整的 last_count。若使用异步的方式调用，则可打断正在执行的运动，并进行新的运动，同时在异步方式下，运动时间为指令下发的间隔时间。

例 1

```
SpeedJ --vel={0.1} --acc={10} --dec={10} --jerk={10} --last_count=1000
delay_ms（950）
SpeedJ --vel={-0.1} --acc={10} --dec={10} --jerk={10} --last_count=1000
SpeedJ --stop
```

五、常用系统指令

1.Clear 错误清理

➢ 指令用法	清除异常等级为 ERROR 的异常，恢复系统到正常状态
➢ 指令参数	无
➢ 指令说明	用于报错等级为 ERROR 时，清除错误信息，并不会清除变量 Var；
➢ 指令示例	无

2.SetTool 设置工具

➢ 指令用法

设置末端工具坐标系

➢ 指令参数

	数据类型	含义
--tool_var	tool	工具坐标系位姿信息

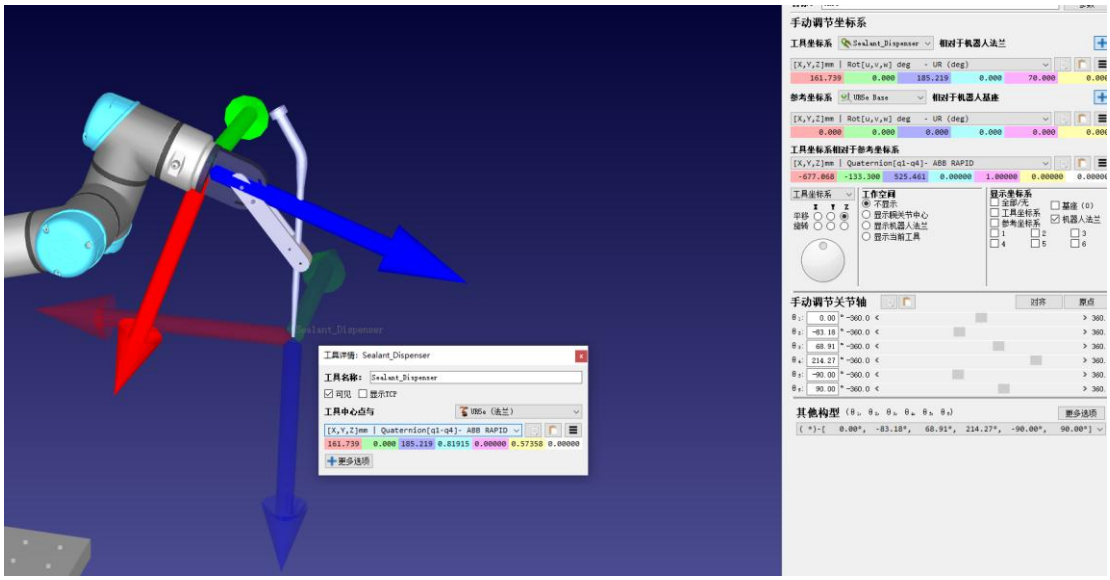
➢ 指令说明

➢ 指令示例

例 1

```
Var --type=tool --name=tool_new --value={0.161739,0,0.185219,0,0.57358,0,0.81915}
SetTool --tool_var=tool_new
```

注意：在末端法兰盘坐标系下，期望的新 tool 的坐标


手动调节坐标系
工具坐标系
Sealant_Dispenser
相对于机器人法兰
[X,Y,Z]mm | Rot[u,v,w] deg - UR (deg)
161.739 0.000 185.219 0.000 70.000
参考坐标系
UR5e Base
相对于机器人基座
[X,Y,Z]mm | Rot[u,v,w] deg - UR (deg)
0.000 0.000 0.000 0.000 0.000
工具坐标系相对于参考坐标系
[X,Y,Z]mm | Quaternion[q1-q4]- ABB RAPID
-677.068 -133.300 525.461 0.00000 1.00000 0.00000
工具坐标系
平移
旋转
工作空间
不显示
显示耦关节中心
显示机器人法兰
显示当前工具
显示坐标系
全部/无
基座 (0)
工具坐标系
机器人法兰
参考坐标系
手动调节关节轴
对齐
原点
其他构型
更多选项
工具详情: Sealant_Dispenser
工具名称: Sealant_Dispenser
可见
显示TCP
工具中心点与
UR5e (法兰)
[X,Y,Z]mm | Quaternion[q1-q4]- ABB RAPID
161.739 0.000 185.219 0.81915 0.00000 0.57358 0.00000
更多选项

3.SetWobj 设置基坐标系

➢ **指令用法**　　设置基坐标系

➢ **指令参数**

	数据类型	含义
-- wobj_var	wobj	基坐标系位姿信息

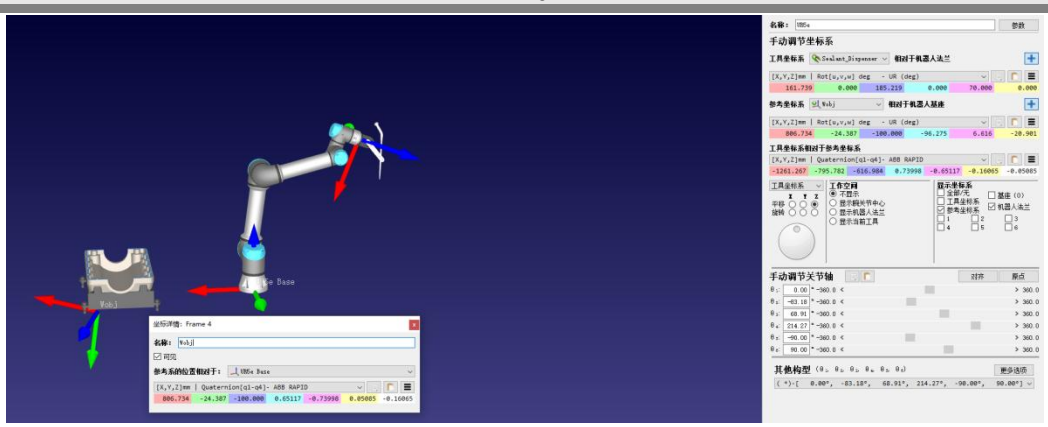
➢ **指令说明**

➢ **指令示例**

例 1

```
Var --type=wobj --name=wobj_new --value={0.392367,-0.024387,0,0,0,0,0}
SetWobj --wobj_var=wobj_new
```

注意：在 base 坐标系下，期望的新 wobj 的坐标



4.SetUsingSP 最优规划求解

➢ **指令用法**　　最优规划求解模式

➢ **指令参数**

数据类型	含义
--state	开关

➢ **指令说明**

➢ **指令示例**

例 1

```
SetUsingSP --state=on
SetUsingSP --state=off
```

六、高级规划指令

可实现直线、圆弧轨迹。七轴情况下需要开启“最优求解”模式, 参考“系统指令”：SetUsingSP。

1.MoveBlend 笛卡尔空间下轨迹融合

➢ **指令用法**

笛卡尔空间下，MoveBlend 用于对相邻两段运动之间的拐点进行**局部轨迹融合**，通过在拐角处引入过渡曲线（如圆弧或平滑段），避免速度突变和频繁启停。

➢ **指令参数**

指令参数	数据类型	含义
--robottarget/ --robottarget_var/ --robottarget_value/ --robottar_pa_value/ --robottar_pm_value/	robottarget	目标点位置变量
--mid_robottarget/ --mid_robottarget_var/ --mid_robottarget_value/	robottarget	中间点位置变量
--speed/ --speed_var/ --speed_value/	Speed	速度变量
--type		插值类型，默认 insert
--zone	zone	轨迹融合变量
--ratio	Number	控制拐点前后轨迹参与融合的相对比例与平滑程度
--is_optimal	Number	当使用 MoveBlendOptimalpos 时，参数设为 1

➢ **指令说明**

1.zone 参数控制融合区宽度，值越大融合越平滑，但目标点误差越大；值越小则越精确但平滑度降低。

2.轨迹点姿态可以变化，执行轨迹时姿态变化也会执行，但有姿态变化时，速度会降低（比较慢）

3.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert_line 直线轨迹点类型，type=insert_circle 圆弧轨迹点类型，type=start 执行的类型（固定的）后面不用加其他参数

4.每段速度可以单独决定，由标记字段指令 speed 决定

➢ **指令示例**

例 1 笛卡尔空间下轨迹融合直线运动

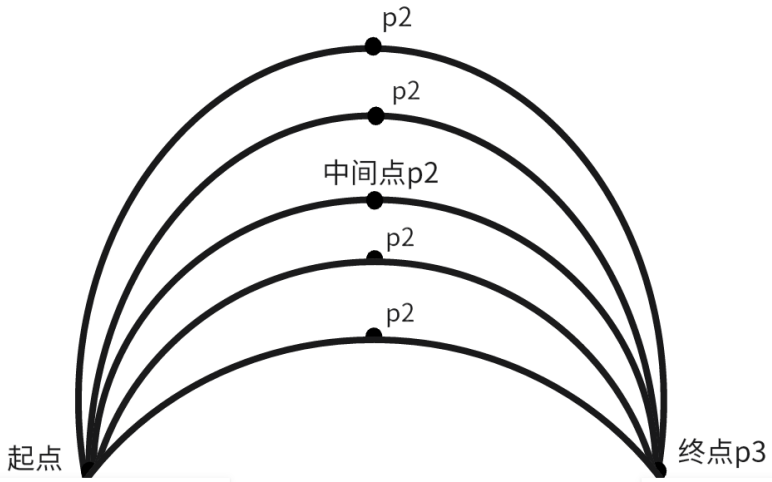
```
Var --type=robottarget --name=p1 --value={0.32, -0.32, 0.48,0, 1, 0, 0};//定义轨迹点位姿
MoveBlend --type=first_insert    //起点
MoveBlend --type=insert_line --robottarget_var=p1 --zone={0.1,0.1} --speed=v300//直线轨迹点标记起点到 p1，zone 越大，可脱离原轨迹程度越大
```

MoveBlend --type=start　//执行命令，速度不由这一行决定，由标志点决定
//其他省略的参数均为默认值

例 2 笛卡尔空间下轨迹融合圆弧运动

```
Var --type=robottarget --name=p2 --value={0.36, -0.28, 0.48,0, 1, 0, 0};
Var --type=robottarget --name=p3 --value={0.40, -0.32, 0.48,0, 1, 0, 0};
MoveBlend --type=first_insert　//起点
MoveBlend --type=insert_circle --mid_robottarget_var=p2 --robottarget_var=p3 --zone={0.1,0.1} --speed=v300
//圆弧轨迹点标记//zone 越大，可脱离原轨迹程度越大，以 p2 作为中间点，圆弧运动到 p3
MoveBlend --type=start　//执行命令，速度不由这一行决定，由标志点决定
//其他省略的参数均为默认值
```

笛卡尔空间下轨迹融合圆弧运动如图下所示



2.MoveBlendScurve 笛卡尔空间下轨迹融合（匀速）

➢ **指令用法**

笛卡尔空间下，机械臂末端执行基于 S 曲线插值算法的融合轨迹运动，用于实现复杂运动中，运动线速度按照制定曲线运动，适用于涂胶、打磨等恒速情形下。

➢ **指令参数**

数据类型	含义
--speed/ --speed_var/ --speed_value/ Speed	速度变量
--acc Number	笛卡尔加速度，单位为 rad/s/s
--jerk Number	笛卡尔加加速度，单位为 rad/s/s/s

➢ **指令说明**

1.zone 参数控制融合区宽度，值越大融合越平滑，但目标点误差越大；值越小则越精确但平滑度降低。

2.轨迹仅执行位置变化，就算轨迹点姿态变化了，也不会执行姿态变化，但也不会报错

3.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert_line 直线轨迹点类型，type=insert_circle 圆弧轨迹点类型，MoveBlendScurve 执行

4.每段速度不可以单独决定，由 MoveBlendScurve 指令 speed 决定

➢ **指令示例**

例 1 笛卡尔空间下轨迹融合直线运动

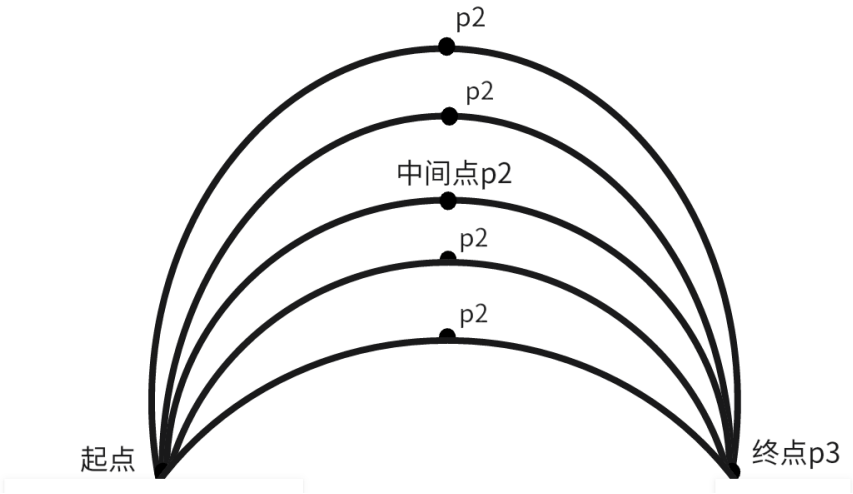
```
Var --type=robottarget --name=p1 --value={0.491, -0.1, 0.687,-0.5, 0.5, -0.5, 0.5};//定义轨迹点位姿
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5};
Var --type=robottarget --name=p3 --value={0.491, 0.02, 0.587，-0.5, 0.5, -0.5, 0.5};
MoveBlend --type=first_insert    //起点
MoveBlend --type=insert_line --robottarget_var=p1 --zone={0.1,0.1}//直线轨迹点标记，zone 越大，可脱离原轨迹程度越大
MoveBlend --type=insert_line --robottarget_var=p2 --zone={0.1,0.1};
MoveBlend --type=insert_line --robottarget_var=p3 --zone={0.1,0.1};
MoveBlendScurve --speed_var=v100 //执行命令，速度仅由这一行决定
//其他省略的参数均为默认值
```

例 2 笛卡尔空间下轨迹融合圆弧运动

```
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5};//定义轨迹点位姿
Var --type=robottarget --name=p3 --value={0.491, 0.02, 0.587，-0.5, 0.5, -0.5, 0.5};
MoveBlend --type=first_insert    //起点
MoveBlend --type=insert_circle --robottarget=p3 --mid_robottarget=p2 --zone={0.01,0.05} /zone 越大，可脱离原轨迹程度越大，以 p2 作为中间点，圆弧运动到 p3
MoveBlendScurve --speed_value=0.1 --acc=0.02 --jerk=0.05//执行命令，速度仅由这一行决定
```

//其他省略的参数均为默认值

笛卡尔空间下轨迹融合圆弧运动如图下所示



3.MoveBlendOptimalpos 笛卡尔空间下轨迹融合（最大速）

➢ **指令用法**

笛卡尔空间下，机械臂末端执行融合轨迹运动，生成的轨迹在满足关节电机最大速度、最大加速度、最大力矩的前提下，尽快的完成运动。

➢ **指令参数**

	数据类型	含义
--acc_coef	Number	加速度系数,值越大,轨迹加速越快
--vel_coef	Number	速度系数，值越大，运行越快
--dynamics_constraint	Number	是否启用机器人动力学约束。1 表示启用（默认），0 表示忽略约束。

➢ **指令说明**

1.zone 参数控制融合区宽度，值越大融合越平滑，但目标点误差越大；值越小则越精确但平滑度降低。

2.轨迹点姿态不要变化，否则会影响到计算，可能会失败

3.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert_line 直线轨迹点类型，type=insert_circle 圆弧轨迹点类型，MoveBlendScurve 执行

4.采用此指令时，标记点字段需要加上--is_optimal=1

➢ **指令示例**

例 1

```
Var --type=robottarget --name=p1 --value={0.491, -0.1, 0.687,-0.5, 0.5, -0.5, 0.5};//定义轨迹点位姿
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5};
Var --type=robottarget --name=p3 --value={0.491, 0.02, 0.587，-0.5, 0.5, -0.5, 0.5};
MoveBlend --type=first_insert //起点标记
MoveBlend --type=insert_line --robottarget_var=p1 --zone={0.1,0.1} --is_optimal=1;//直线轨迹点标记
MoveBlend --type=insert_line --robottarget_var=p2 --zone={0.1,0.1} --is_optimal=1;
MoveBlend --type=insert_line --robottarget_var=p3 --zone={0.1,0.1} --is_optimal=1;//采用示教点位时，加上这个参数
MoveBlendOptimalPos --acc_coef={0.9} --vel_coef={0.9} --dynamics_constraint=0;
//执行命令
//其他省略的参数均为默认值
```

4.MoveS 笛卡尔空间下轨迹融合（全程）

➢ **指令用法**

笛卡尔空间下，用于对多个路径点进行整体样条拟合，生成一条全局连续的平滑轨迹，属于全局融合

➢ **指令参数**

指令参数	数据类型	含义
--robottarget/ --robottarget_var/ --robottarget_value/ --robottarget_pa_value/ --robottarget_pm_value/	robottarget	目标点位置变量
--speed/ --speed_var/ --speed_value/	Speed	速度变量
--type		插值类型，默认 insert
--ratio	Number	全局平滑权重，决定平滑程度

➢ **指令说明**

1.轨迹点姿态可以变化，执行轨迹时姿态变化也会执行，但有姿态变化时，速度会降低（比较慢）

2.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert 轨迹点标记，type=start 执行

3.每段速度可以不单独决定，由 MoveS --type=start 后 speed 决定

➢ **指令示例**

例 1

```
Var --type=robottarget --name=p1 --value={0.491, -0.1, 0.687,-0.5, 0.5, -0.5, 0.5}//定义轨迹点位姿
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5}
Var --type=robottarget --name=p3 --value={0.491,0.02, 0.587,-0.5, 0.5, -0.5, 0.5}
MoveS --type=first_insert    //起点
MoveS --type=insert --robottarget_var=p1//轨迹点标记
MoveS --type=insert --robottarget_var=p2//轨迹点标记
MoveS --type=insert --robottarget_var=p3//轨迹点标记
MoveS --type=start --speed=v200    //执行命令，速度仅由这一行决定，不由轨迹点速度决定
//其他省略的参数均为默认值
```

5.ServoSeries 关节空间下轨迹融合指令

➢ **指令用法**

关节空间连续运动，连续插值轨迹控制指令，用于向控制器发送一系列关节空间目标点，实现机械臂轨迹的平滑执行，适用于实时控制和柔顺运动场景。

➢ **指令参数**

	数据类型	含义
--joint_set	jointtarget	需要完成的期望关节位置列表
--time	Number	需要完成对应期望关节位置的时间列表
--stop_time	Number	在完成所有关节位置后留下的减速时间
--scale	Number	时间缩放比例，设置的时间列表*scale 是最终执行的事件列表

➢ **指令说明**

用户设置一系列的期望关节位置和其对应的时间列表，指令对于不同关节位置间进行插值，在设置的时间点完成对应的期望关节位置。

➢ **指令示例**

例 1 中，设置了 4 组期望关节位置，首先需要先运动到起始点 J11，然后开始 servoseries 指令，缩放比例为 1，先花费 2 秒到达 J12，再花费 2 秒到达 J13，再花费 4 秒到达 J14。

例 1

```
Var --type=jointtarget --name=j11 --value={0,0,0,1.4573,0,-1.57,0,0,0,0}
Var --type=jointtarget --name=j12 --value={-0.8,0,0,0.1,0,-1.57,0,0,0,0}
Var --type=jointtarget --name=j13 --value={-0.8,1.57,0,0.1,0,-1.57,0,0,0,0}
Var --type=jointtarget --name=j14 --value={0,0,0,0.1,0,0,0,0,0,0}
MoveAbsJ --jointtarget_var=j11
ServoSeries --joint_set={j11,j12,j13,j14} --time={1,3,5,9} --scale=1
```

6. MoveSeriesToppJ2 关节空间下轨迹融合（最大速）

➢ **指令用法**

关节空间下，用于以关节空间轨迹的形式，按照时间最优方式执行一系列关节点，确保在速度和加速度约束下，实现最快的轨迹跟踪。

➢ **指令参数**

	数据类型	含义
-- jointtarget	jointtarget	需要完成的期望关节位置列表
--jointtarget_var/ --jointtarget_value		
--type		插入类型
--acc_coef	Number	加速度系数，范围为(0, 1],值越大,轨迹加速越快 (默认 0.95)
--vel_coef	Number	速度系数，范围为(0, 1]，值越大，运行越快 (默认 0.95)

➢ **指令说明**

1.轨迹点应事先设定为顺序连续的姿态/关节数据，避免大角度跳变；
2.点之间不能出现突变或不可达配置，否则插值会失败。

➢ **指令示例**

例 1

```
Var --type=jointtarget --name=j1 --value={0.0,-1.57,-1.57,-1.57,1.57,0.0,0,0,0,0}
Var --type=jointtarget --name=j2 --value={0.0,-1.57,-1.13,-1.57,1.57,0.0,0,0,0,0}
Var --type=jointtarget --name=j3 --value={0.52,-1.57,-1.13,-1.57,1.57,0.0,0,0,0,0}
MoveSeriesToppJ2 --type=first_insert
MoveSeriesToppJ2 --type=insert –jointtarget_var=j1
MoveSeriesToppJ2 --type=insert --jointtarget_var =j2
MoveSeriesToppJ2 --type=insert --jointtarget_var =j3
MoveSeriesToppJ2 --type=start
```

例 2

```
MoveSeriesToppJ2 --type=first_insert
MoveSeriesToppJ2 --type=insert –jointtarget_value={0.0,-1.57,-1.57,-1.57,1.57,0.0,0,0,0,0}
MoveSeriesToppJ2 --type=insert --jointtarget_value ={0.0,-1.57,-1.13,-1.57,1.57,0.0,0,0,0,0}
MoveSeriesToppJ2 --type=insert --jointtarget_value={0.52,-1.57,-1.13,-1.57,1.57,0.0,0,0,0,0}
MoveSeriesToppJ2 --type=start
```

七、力控指令

1. DragFtInCartesian 笛卡尔空间下力-位混合控制

➢ **指令用法**

笛卡尔空间下，机器人末端沿设定方向感受到外力时做出运动响应，形成柔顺拖动控制（基于六维力的拖动示教），开启位置控制后，可以让机器人在移动过程中具备柔顺性，对外力进行反馈。

➢ **指令参数**

	数据类型	含义
--force_direction	Matrix	允许进行拖动的自由度（方向），1 表示启用，0 表示禁用
--damping_gain	Matrix	各个自由度（方向）拖动示教灵敏度参数，值越大越易拖动
--vel_limit	Matrix	拖动过程中的速度限制
--acc_limit	Matrix	拖动过程中的加速度限制
--open_tg	Matrix	1 表示启用力-位置混合控制，0 表示禁用
--coordinate	Number	拖动时参考坐标系，1 为工具坐标系（默认），0 为基坐标系
--cutoff_freq	Number	六维力传感器数据一阶低通滤波器
--tool/--tool_var	tool	工具坐标系
--wobj/--wobj_var	wobj	工件坐标系

➢ **指令说明**

要将六维力传感器坐标系与工具坐标系对齐，否则方向可能会不对

➢ **指令示例**

例 1 中，开启 z 方向的拖动，给传感器 z 方向的力，末端就会朝着相应的方向移动。

例 2 中，直线运动到 p1，在运动过程中开启 y 方向的力控且期望力为 6

例 1

```
 SetUsingSP --state=on //开启最优求解器功能

DragFtInCartesian --force_direction={0,0,1,0,0,0} --damping_gain={0.03,0.03,0.03,0.3,0.3,0.5}
    //其它参数可省略采用默认值
```

例 2

```
Var --type=robottarget --name=p1 --value={0.490,0.1,0.680,0,0.70710678,0,0.70710678}

MoveBlend --type=first_insert
MoveBlend --type=insert_line --robottarget=p1
DragFtInCartesian --force_target={0,0,6,0,0,0} --force_direction={0,0,1,0,0,0} --open_tg=1
```

八、IO 模块指令

1.GetDI 读取输入电平状态

➢ **指令用法** 读取 IO 模块的指定数字输入（DI）通道的实时电平状态

➢ **指令参数**

	数据类型	含义
--di_name	String	设备通道号

➢ **指令说明** 该指令仅返回当前 DI 引脚的电平状态不改变硬件状态

➢ **指令示例**

例 1

```
GetDI --do_name=DO0; //读取数字通道 DO0 上的电平状态
```

2.SetDO 设置输出电平状态

➢ **指令用法**　设置指定数字输出（DO）通道的实时电平状态

➢ **指令参数**

	数据类型	含义
--do_name	String	设备通道号
--do_value	Number	目标输出电平逻辑状态

➢ **指令说明**　该指令为同步写入操作，执行后 DO 引脚电平即刻变化

➢ **指令示例**

例 1

```
SetDO --do_name=DO0 --do_value=0; //设置数字通道 DO0 上的电平状态为 0 状态
```

3.PulseDO 输出数字脉冲信号

➢ **指令用法** 设置指定数字输出（DO）通道输出数字脉冲信号

➢ **指令参数**

	数据类型	含义
--do_name	String	设备通道号
--pulse_active	Number	是否启用脉冲输出
--high_cycles	num	高电平持续周期数
--low_cycles	num	低电平持续周期数

➢ **指令说明** --do_name 必须是已定义的有效 DO 通道名

➢ **指令示例**

例 1

PulseDO --do_name=DO0 --pulse_active=1 --high_cycles=10 --low_cycles=10; //数字输出通道 DO0 上输出一个由高电平和低电平组成的、可配置时长的脉冲序列

4.AddIO 注册新的 I/O 信号输出口

➢ **指令用法**　注册指定数字输入输出（DI/O）通道口

➢ **指令参数**

	数据类型	含义
--io_type	String	指定要添加的 I/O 信号类型
--io_num	String	物理通道编号
--io_name	String	变量名

➢ **指令说明**

--io_num 是硬件层地址；
--io_name 是软件层标识符

➢ **指令示例**

例 1

AddIO --io_type=DO --io_num=1 --io_name="EMERGENCY_STOP";// 将物理 DO1 通道注册为变量"EMERGENCY_STOP"